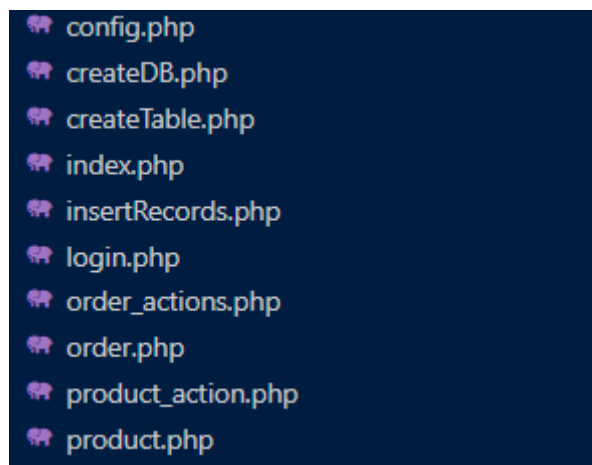


About Project

My goal in this project is to manage a store. The sales person can enter their information and add products and orders. While every salesperson can see the products, only the person who made the sale can see the sales information. We're creating a simple and fast interface program that's suitable for real store order management. I was inspired by a real project and designed it with my own ideas. My experience in commerce also contributed to my design. We use search buttons to get detailed information about our products and orders. For the database, we used MySQL via phpMyAdmin.

Projects codes



`createdDB.php`: This file is used to set up the database named "store" that we designed for our project. We run `createdTable.php` to set up the tables.

We use `config.php` to quickly access the database on other pages. Here, we access the database using constant variables.

To add some data based on our example project, we run `insertRecords.php`. Then our database is ready with the inserted values. After verifying user logins with our `login.php` page, we redirect to our main page, `index.php`. From our main page, we can access our product and order pages.

The `Order` and `Product` action files do not show any page code. They are a structure I designed based on the example project for our database operations.

Database

```
$users_table = "  
CREATE TABLE users (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  username VARCHAR(50) NOT NULL UNIQUE,  
  password VARCHAR(255) NOT NULL  
)";
```

	store users
🔑	id : int
🔑	username : varchar(50)
📄	password : varchar(255)

We assigned a unique "username" to each user, so we didn't add any additional information. Additional information, such as a "phone number," can be added if needed.

```
$products_table = "  
CREATE TABLE products (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  product_name VARCHAR(100) NOT NULL UNIQUE,  
  price DECIMAL(10,2) NOT NULL  
)";
```

	store products
🔑	id : int
🔑	product_name : varchar(100)
#	price : decimal(10,2)

We should think of our products' names as their "modal name." This way, for each product, we keep it as product_name in addition to the ID. It is a short name that gives us quick information about the product. We will connect the price to orders because this price will be recorded as the item price in orders.

```
$orders_table = "  
CREATE TABLE orders (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  order_date DATETIME NOT NULL,  
  created_by INT NOT NULL,  
  FOREIGN KEY (created_by) REFERENCES users(id)  
)";
```

	store orders
🔑	id : int
📅	order_date : datetime
#	created_by : int

Here, we will connect with the items table for the normalization model. We use created_by to make each user create a separate order and to link seller and order. If our seller is deleted, the order will not be deleted, and this is very important.

```

    $items_table = "
CREATE TABLE items (
    id INT AUTO_INCREMENT PRIMARY KEY,
    order_id INT NOT NULL,
    product_id INT NOT NULL,
    quantity INT NOT NULL,
    price DECIMAL(10,2) NOT NULL DEFAULT 0.00
    FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE CASCADE,
    FOREIGN KEY (product_id) REFERENCES products(id)
);

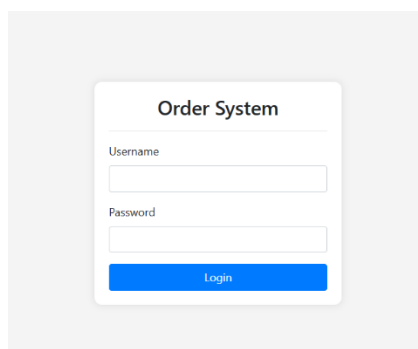
```

v	store items
🔑	id : int
#	order_id : int
#	product_id : int
#	price : decimal(10,2)
#	quantity : int

In this table, we connect with the order table. Our prices are matched with the product price at the time of the order. We use this table in detail to record items in multiple orders. In addition, if an order is deleted, its items are also deleted.

Pages

Login Page – login.php



The image shows a login form titled "Order System". It contains two input fields: "Username" and "Password". Below the password field is a blue button labeled "Login". The form is centered on a light gray background.

The system is based on session management. This means that when a user joins the system, the browser recognizes them, and instead of asking for a password on every page, it uses this secure session.

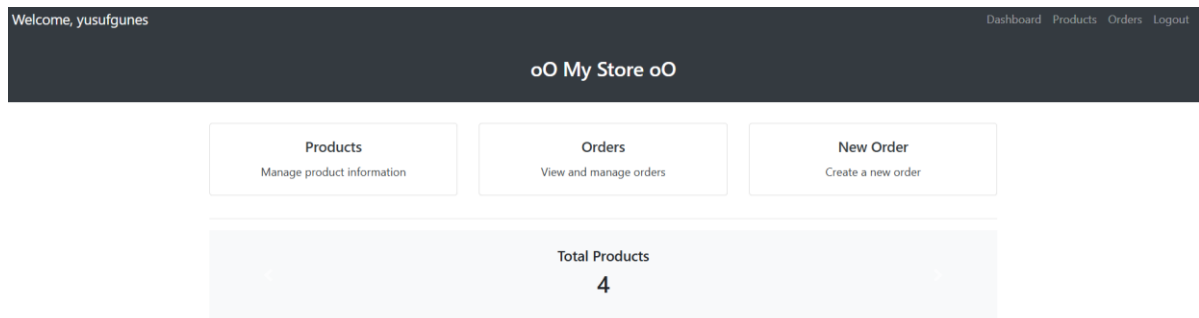
On the database side, I check the username and password immediately. If the information is correct, I send the user directly to the Dashboard where they can do their work. If the information is wrong, I show an error message without refreshing the page.

For security, I used Prepared Statements. This stops attacks like SQL injections before they can reach the database.

For the interface, I used Bootstrap to make a clean login box that looks good and works well on mobile devices. I also added filters in the code to prevent browser-based attacks like XSS.

In the end, the result is a fast and secure login system that keeps data safe.

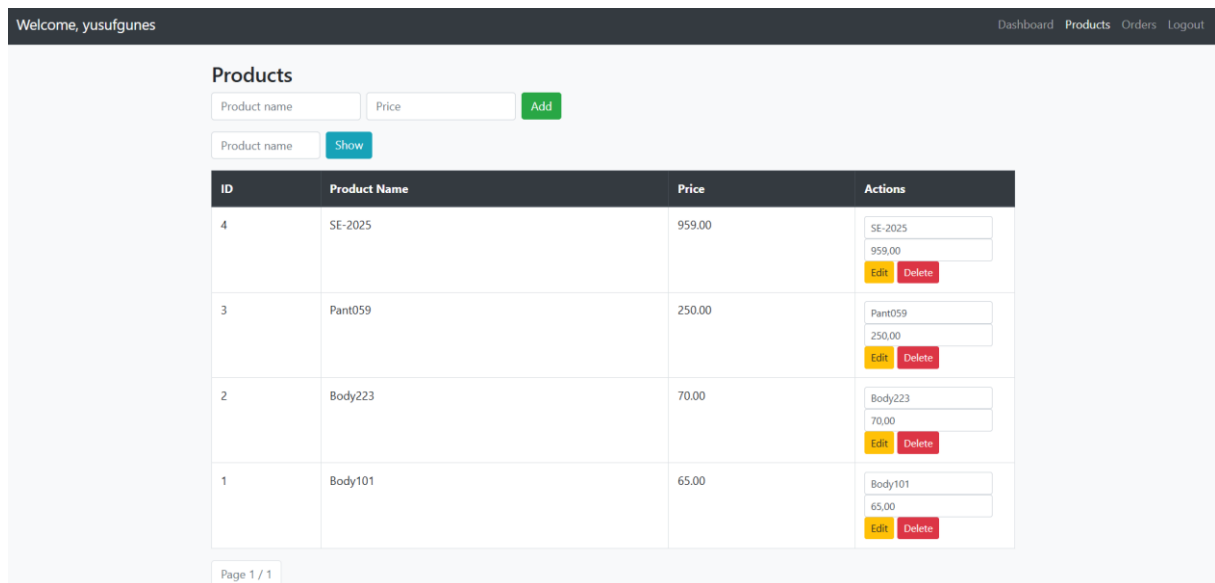
Main Page – index.php



Our overall design is simple and fast. The menu is available on every page, so we can move between pages quickly. We added two active salespeople who can log in. The logged-in seller's username appears at the top left to greet us.

We navigate between pages using four links. For important sections, we created large links using "div". At the bottom, there is an automatic sliding panel that shows the total number of orders and products in detail as a slideshow.

Product page – product.php



Here, we have a table about products. All products are shown in the table in order from newest to oldest. Any seller can add products, and they are not specific to a seller. The "Product Name" column should be seen as "modal codes" for the products. This is actively used in e-commerce marketplaces.

We use pagination for fast navigation. Each page shows 10 products. To see other products, we use the "Next" and "Previous" buttons. It is flexible and fast.

Next to our products, we use “Edit” to quickly update them. Here, let’s update the price of our product “SE-2025” and check the database. We can also see the update in our table.

	id	product_name	price	
☐	Düzenle	Kopyala	Sil	1 Body101 65.00
☐	Düzenle	Kopyala	Sil	2 Body223 70.00
☐	Düzenle	Kopyala	Sil	3 Pant059 250.00
☐	Düzenle	Kopyala	Sil	4 SE-2025 969.90

4	SE-2025	969.90	SE-2025 969,90 EditDelete
---	---------	--------	---

When we use the “Delete” button, first we show the user an alert message: “Are you sure?”

localhost şunu diyor:

Delete product?

Tamam İptal

ID	Product Name	Price	Actions
4	SE-2025	969.90	SE-2025 969,90 EditDelete
2	Body223	70.00	Body223 70,00 EditDelete
1	Body101	65.00	Body101 65,00 EditDelete

Page 1 / 1

	id	product_name	price	
☐	Düzenle	Kopyala	Sil	1 Body101 65.00
☐	Düzenle	Kopyala	Sil	2 Body223 70.00
☐	Düzenle	Kopyala	Sil	4 SE-2025 969.90

With the two inputs at the top, we can quickly add a product.

With the input below, we can type the product name to get information about it. This is important for companies that have many products.

Products

Product name Price

SE-2025 Show

localhost şunu diyor:

ID: 4
ProductName: SE-2025
Price: 969.90 TL

Tamam

Order Page – order.php

The screenshot shows the 'Orders' section of a dashboard. At the top, there's a header with 'Welcome, yusufgunes' and navigation links for 'Dashboard', 'Products', 'Orders', and 'Logout'. Below the header, the 'Orders' section has a '+ Create Order' button and a search bar with 'Order ID' and a 'Show Order Information' button. A table lists four orders with columns for ID, Date, Created By, and Actions. Each order has 'View' and 'Delete' buttons.

ID	Date	Created By	Actions
17	2025-12-28 18:27:32	yusufgunes	View Delete
14	2025-12-28 18:27:15	yusufgunes	View Delete
13	2025-12-28 18:27:15	yusufgunes	View Delete
12	2025-12-28 18:27:13	yusufgunes	View Delete

First, we should click the “Create Order” button. When we click on an order, I immediately fetch and combine data from different tables to see which user the order belongs to, which products are inside, and the current names of those products.

The screenshot shows the 'Order #12 (yusufgunes)' details panel. It has a table with columns for Product, Quantity, Price, and Action. The table lists two products: SE-2025 and F1567, each with a quantity of 2 and a price. Below the table, there's a search bar with 'Product name' and 'Qty' and an 'Add Item' button.

Product	Quantity	Price	Action
SE-2025	2	969,90	Edit Delete
F1567	2	1259,00	Edit Delete

I designed the page with a two-step logic. In the first step, I show all the user’s past orders in a summary table. When the user clicks the “View” button, a “Cart Details” panel opens below for that specific order. In this panel, the user can update the quantity and price of ordered products, add new products, or remove existing products. So, this is basically a live order editing panel. For deleting items or orders, we show a warning message.

On the security side, I use Prepared Statements for all database operations to prevent attacks through order IDs. Also, the smart search engine I added communicates with order_actions.php using the JavaScript fetch API. When a user enters an order number, the page shows critical information like total amount, customer name, and product list instantly in a popup without refreshing the page.

SİPARİŞ NO: #12
Date: 2025-12-28 18:27:13
SELLER: yusufgunes

PRODUCTS:
- SE-2025 x 2 | 969.90 TL
- F1567 x 2 | 1,259.00 TL

TOTAL PRICE: 4,457.80 TL

Tamam

With the “Show Order Information” button, we get very detailed information about an order. Here, we can see which products were bought, who sold them, when the order was made, and the quantities and total price of the products.

localhost şunu diyor:

SİPARİŞ NO: #4
Date: 2025-12-28 18:09:05
SELLER: mertkartepe

PRODUCTS:
- Body223 x 12 | 70.00 TL
- SE-2025 x 2 | 959.00 TL

TOTAL PRICE: 2,758.00 TL

Tamam

ID	Date	Actions
15	2025-12-28 18:27:23	View Delete
8	2025-12-28 18:09:27	View Delete
7	2025-12-28 18:09:27	View Delete
4	2025-12-28 18:09:05	View Delete

Order #4 (mertkartepe)

Product	Quantity	Price	Action
Body223	12	70.00	Edit Delete
SE-2025	2	959.00	Edit Delete

Product name Qty [Add Item](#)

Actions – order_action.php & product_action.php

On our example project page, we perform reading, deleting, and updating operations. We use “if” statements to make decisions based on some text sent from the main page.

Database Records

User table

				id	username	password
☐		Düzenle		Kopyala		Sil
				1	mertkartepe	1234
☐		Düzenle		Kopyala		Sil
				2	yusufgunes	123

Product Table

					id	product_name	price
☐		Düzenle		Kopyala		Sil	
					1	Body101	65.00
☐		Düzenle		Kopyala		Sil	
					2	Body223	70.00
☐		Düzenle		Kopyala		Sil	
					5	F1567	1259.00
☐		Düzenle		Kopyala		Sil	
					4	SE-2025	969.90
☐		Düzenle		Kopyala		Sil	
					6	F1608	2199.00
☐		Düzenle		Kopyala		Sil	
					7	EK-1	2599.00
☐		Düzenle		Kopyala		Sil	
					8	EK-2	1799.00
☐		Düzenle		Kopyala		Sil	
					9	1123	1359.00
☐		Düzenle		Kopyala		Sil	
					10	1551	959.00
☐		Düzenle		Kopyala		Sil	
					11	801	1599.00
☐		Düzenle		Kopyala		Sil	
					12	823	279.00
☐		Düzenle		Kopyala		Sil	
					13	Body129	123.00

Order Table

					id	order_date	created_by
☐		Düzenle		Kopyala		Sil	
					14	2025-12-28 18:27:15	2
☐		Düzenle		Kopyala		Sil	
					13	2025-12-28 18:27:15	2
☐		Düzenle		Kopyala		Sil	
					4	2025-12-28 18:09:05	1
☐		Düzenle		Kopyala		Sil	
					7	2025-12-28 18:09:27	1
☐		Düzenle		Kopyala		Sil	
					8	2025-12-28 18:09:27	1
☐		Düzenle		Kopyala		Sil	
					12	2025-12-28 18:27:13	2
☐		Düzenle		Kopyala		Sil	
					15	2025-12-28 18:27:23	1
☐		Düzenle		Kopyala		Sil	
					17	2025-12-28 18:27:32	2

Items Table

← T →					id	order_id	product_id	price	quantity		
		Düzenle		Kopyala		Sil	10	4	2	70.00	12
		Düzenle		Kopyala		Sil	11	7	4	959.00	4
		Düzenle		Kopyala		Sil	13	4	4	959.00	2
		Düzenle		Kopyala		Sil	14	12	4	969.90	4

Finally